"Concurrent Producer-Consumer System Using Semaphores"

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE & INFORMATION TECHNOLOGY

By

V. SUJIT	**2520090085**
D. BALA CHARAN	**2520090087**
M. SAI CHAITANYA	**2520090023**

Under the Esteemed Guidance of

Dr. M Raghupathi

Assistant Professor

Department of Computer Science and Information Technology



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



Declaration

The Project Report entitled **"Concurrent Producer-Consumer System Using Semaphores"** is a record of Bonafide work of **V. SUJIT – 2520090085, D. BALA CHARAN - 2520090087, M. SAI CHAITANYA - 2520090023** submitted in partial fulfillment for the award of B. Tech in Computer Science and Information Technology to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

V. SUJIT	**2520090085**
D. BALA CHARAN	**2520090087**
M. SAI CHAITANYA	**2520090023**

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING



CERTIFICATE

This is certify that the mini project based report entitled **"Concurrent Producer-Consumer System Using Semaphores"** is a bonafide work done and submitted by **V. SUJIT – 2520090085, D. BALA CHARAN - 2520090087, M. SAI CHAITANYA – 2520090023** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science and Information Technology, K L (Deemed to be University), during the academic year **2026-2027.**

Signature of the Guide

Signature of the Course Coordinator **Signature of the HOD**

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express my sincere thanks to all those who has assisted us in one way or the other for the completion of my project.

Our greatest appreciation to my Course Coordinator Dr. K. Anusha, and my guide **Dr. M Raghupathi,** Department of Computer Science and Information Technology which cannot be expressed in words for his tremendous support, encouragement and guidance for this project.

We express our gratitude to **Dr. Khaja Shareef (CS&IT)**, Head of the ***Department for Computer Science and Information Technology*** for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and also who have assisted me directly or indirectly for successful completion of this project. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

V. SUJIT **2520090085**
D. BALA CHARAN **2520090087**
M. SAI CHAITANYA **2520090023**

Abstract

In modern multitasking operating systems, coordinating concurrent execution across multiple threads without data corruption is a critical challenge. The Producer–Consumer problem is a classic synchronization paradigm where multiple producer threads generate data into a bounded buffer while consumer threads simultaneously extract it. Without adequate synchronization mechanisms, concurrent access leads to race conditions, buffer overflow, buffer underflow, and deadlocks.

This project implements a robust, multithreaded Producer–Consumer system in a Linux environment using C and POSIX threads (pthreads). To guarantee thread safety and prevent race conditions, the system utilizes POSIX semaphores (sem_t) and mutex locks. Specifically, counting semaphores track the counts of empty and filled slots in the shared circular buffer, blocking producers when the buffer is full and blocking consumers when it is empty. A binary semaphore enforces mutual exclusion during critical section access, ensuring only one thread modifies the buffer at a time. The expected outcome is a deadlock-free, high-performance concurrent application on Linux that demonstrates seamless synchronization, optimal CPU utilization, and verified data integrity.

Index

S. No.	Chapter	Topics / Subtopics	Page No.
	Acknowledgement		
	Abstract		
1	Introduction		1–2
		1.1 Background of the Project	
		1.2 Problem Statement	
		1.3 Scope of the Project	
		1.4 Objectives of the Project	
		1.5 Importance of the Application	
		1.6 Target Users / Audience	
2	OS Concepts and Background Study		3–4
		2.1 Concurrency and Multithreading	
		2.2 Critical Section and Mutual Exclusion	
		2.3 POSIX Semaphores	
		2.5 Race Condition Avoidance	
		2.6 Deadlock Avoidance	
		2.7 Existing / Unsynchronized Approach	
		2.8 Proposed Approach	
		2.9 Advantages of the Proposed Approach	
3	System Requirements		5
		3.1 Hardware Requirements	
		3.2 Software Requirements	
		3.3 Operating System Environment – Ubuntu / WSL	
		3.4 Development Tools and Compilers	
4	Technology and Tools Used		6
		4.1 C Programming	
		4.2 GCC Compiler	
		4.3 POSIX Threads	
		4.4 POSIX Semaphores	
		4.5 POSIX Mutex	
		4.6 Linux Commands and Utilities	

5	**System Architecture and Design**		**7–9**
		5.1 Overall System Architecture	
		5.2 System Workflow	
		5.3 Buffer State Model	
		5.4 Module Description	
		5.5 Input and Output Design	
6	**Implementation**		**10–11**
		6.1 Module-wise Implementation	
		6.2 Algorithm / Pseudocode - Initialization	
		6.3 Algorithm / Pseudocode - Producer	
		6.4 Algorithm / Pseudocode - Consumer	
		6.5 Linux System Calls / APIs Used	
		6.6 Process / Thread Implementation	
		6.7 Synchronization Implementation	
		6.8 Memory Management Implementation	
7	**Execution and Demonstration**		**12–14**
		7.1 Compilation in Ubuntu / Linux	
		7.2 Sample Execution Sequence	
		7.3 Test Cases	
		7.4 Demonstration of OS Concepts	
		7.5 Validation and Debugging	
		7.6 Screenshots	
8	**Conclusion**		**15**
		8.1 Summary of the Project	
		8.2 Objectives Achieved	
		8.3 Results Achieved	
		8.4 Skills and Learning Outcomes	
		8.5 Limitations and Future Work	
9	**References**		**16**
		9.1 References	

CHAPTER -1 INTRODUCTION

1.1 Background of the Project

A process normally has its own address space. One process cannot directly read another process's ordinary variables. The operating system therefore provides IPC mechanisms through which processes can exchange information. Examples include pipes, named pipes, message queues, shared memory, sockets, and signals.

The Inter-Process Chat Application applies IPC to a familiar use case: a two-person chat. Each participant runs the program in a separate terminal. Messages typed by one participant are written to a FIFO and read by the other participant. Unlike a network chat system, this version focuses on communication between processes on the same Linux machine.

1.2 Problem Statement

Develop a reliable, easy-to-demonstrate C application in which two independent processes can exchange messages in both directions on Ubuntu Linux. The application must use a genuine operating-system IPC mechanism, handle normal termination, and provide clear build and execution instructions.

1.3 Scope of the Project

- The initial version supports two participants on the same Ubuntu computer.
- The interface is a command-line terminal; a graphical interface is outside the current scope.
- The chat is a learning prototype, not a production messaging service.
- Messages are not encrypted or stored in a persistent chat history.
- The FIFO paths and session naming must be consistent between both participants.

1.4 Objectives of the Project

- Understand the purpose and operation of Inter-Process Communication.
- Create and use POSIX named pipes to exchange data between independent processes.
- Support bidirectional messaging using separate communication channels.
- Demonstrate Linux file descriptors and blocking/non-blocking I/O concepts.
- Handle user exit, peer disconnection, and common system-call errors.
- Document compilation, execution, test scenarios, limitations, and future improvements.

1.5 Importance of the Application

The project provides a practical demonstration of how operating-system synchronization primitives control concurrent access to shared resources. It connects theoretical concepts such as race conditions, critical sections, blocking, and deadlock avoidance with a concrete Linux multithreaded application.

1.6 Target Users / Audience

The primary audience is students and learners studying operating systems, systems programming, concurrent programming, POSIX threads, and synchronization. The project can also serve as a compact demonstration for understanding bounded-buffer coordination in a Linux environment.

CHAPTER -2 OS Concepts and Background Study

2.1 Concurrency and Multithreading

Concurrency allows multiple execution flows to make progress during overlapping periods. In this project, producer and consumer activities execute as separate POSIX threads. Because the threads share the bounded buffer, their operations must be coordinated so that a thread never observes or modifies an invalid buffer state.

2.2 Critical Section and Mutual Exclusion

The critical section is the portion of the producer or consumer workflow that accesses the shared circular buffer and its tracking pointers. A mutex lock is used so that only one thread at a time performs the protected update. This prevents simultaneous modification of shared state and reduces the possibility of race conditions.

2.3 POSIX Semaphores

The project uses POSIX semaphores represented by sem_t. Two counting semaphores model resource availability: empty represents the number of unused buffer slots and full represents the number of occupied slots. A producer waits on empty before inserting an item and posts full after insertion. A consumer waits on full before removing an item and posts empty after removal.

2.4 Circular Bounded Buffer

The circular buffer is a fixed-capacity shared array with read and write positions. When a position reaches the end of the array, the corresponding pointer wraps around to the beginning. This provides bounded storage while allowing the producer and consumer workflows to continue using the same fixed set of locations.

2.5 Race Condition Avoidance

A race condition occurs when the correctness of shared data depends on the uncontrolled order in which concurrent threads execute. The project avoids this by combining counting semaphores for buffer-state coordination with a mutex for exclusive access to the shared buffer during the critical section.

2.6 Deadlock Avoidance

The proposed workflow establishes a simple synchronization order: a producer first waits for an empty slot, then enters the protected critical section; a consumer first waits for an available item, then enters the protected critical section. The intended design avoids uncontrolled lock acquisition and uses semaphore signaling to wake blocked participants.

2.7 Existing / Unsynchronized Approach

A basic unsynchronized implementation would allow producers and consumers to access the shared buffer directly. Under concurrent execution, such an approach can produce inconsistent pointers, overwriting of valid data, reading of unavailable data, and other boundary violations. Busy-waiting can also waste CPU time while a thread repeatedly checks whether a condition has changed.

2.8 Proposed Approach

The proposed approach combines pthread_create()/pthread_join() for thread lifecycle management, POSIX counting semaphores for buffer-state coordination, and a mutex for mutual exclusion. This allows blocked threads to wait for the required resource rather than repeatedly polling the shared buffer.

2.9 Advantages of the Proposed Approach

- Prevents producers from inserting into a full buffer.
- Prevents consumers from removing from an empty buffer.
- Protects the shared circular buffer from simultaneous modification.
- Reduces CPU-intensive busy-waiting through blocking semaphore operations.
- Provides a clear and testable demonstration of operating-system synchronization.

CHAPTER 3 – SYSTEM REQUIREMENTS

3.1 Hardware Requirements

The supplied project abstract does not specify a minimum hardware configuration. The application is intended to run in a Linux environment capable of compiling and executing a C program with POSIX threads. Exact hardware requirements may therefore be recorded after the implementation environment is finalized.

Component	Requirement / Note
Processor	Multicore or standard Linux-capable processor; exact model not specified in the abstract.
Memory	Sufficient RAM for the Linux environment and concurrent threads; exact value not specified.
Storage	Space for source code, executable, logs, and debugging output.
Input/Output	Terminal/CLI environment for compilation and execution.

3.2 Software Requirements

Software / Library	Purpose
Linux / Ubuntu	Primary development, multithreading execution, and testing environment.
C / C++ toolchain	System-level programming; the project description identifies C and GCC compilation.
POSIX Threads Library (pthreads)	Thread creation and synchronization API.
POSIX Semaphores	sem_init, sem_wait, and sem_post for coordination.
GDB & Valgrind (Helgrind)	Debugging and verification of race conditions, locks, and thread safety.
Git & GitHub	Version control and collaborative source-code management.

3.3 Operating System Environment

Linux/Ubuntu is the primary execution environment stated in the project material. The use of POSIX threads, POSIX semaphores, mutexes, and GCC is aligned with the intended Linux systems-programming environment.

3.4 Development Tools and Compiler

The abstract identifies GCC as the compiler environment and specifies pthread-based compilation with the pthread library. GDB and Valgrind/Helgrind are identified for debugging and thread-safety verification.

```
gcc producer_consumer.c -o producer_consumer -lpthread
./producer_consumer
```

The command above represents the compilation pattern implied by the project description. The final source filename and executable name should be replaced with the names used by the team.

CHAPTER 4 – TECHNOLOGY AND TOOLS USED

4.1 C Programming

C is used for system-level implementation of the circular buffer, worker routines, synchronization logic, and program lifecycle. Its low-level memory and concurrency interfaces are suitable for demonstrating operating-system mechanisms.

4.2 GCC Compiler

GCC is the compiler identified in the project material. The pthread library is linked during compilation so that the POSIX thread APIs are available to the application.

4.3 POSIX Threads

The pthread_create() function is used to spawn producer and consumer worker threads, while pthread_join() is used to synchronize completion before cleanup.

4.4 POSIX Semaphores

sem_init() initializes the synchronization objects. sem_wait() blocks a thread until the required count is available, and sem_post() increments the semaphore and can release a waiting participant.

4.5 POSIX Mutex

The mutex provides exclusive access to the shared circular buffer during the critical section. The project material identifies pthread_mutex_lock() and pthread_mutex_unlock() as the relevant operations.

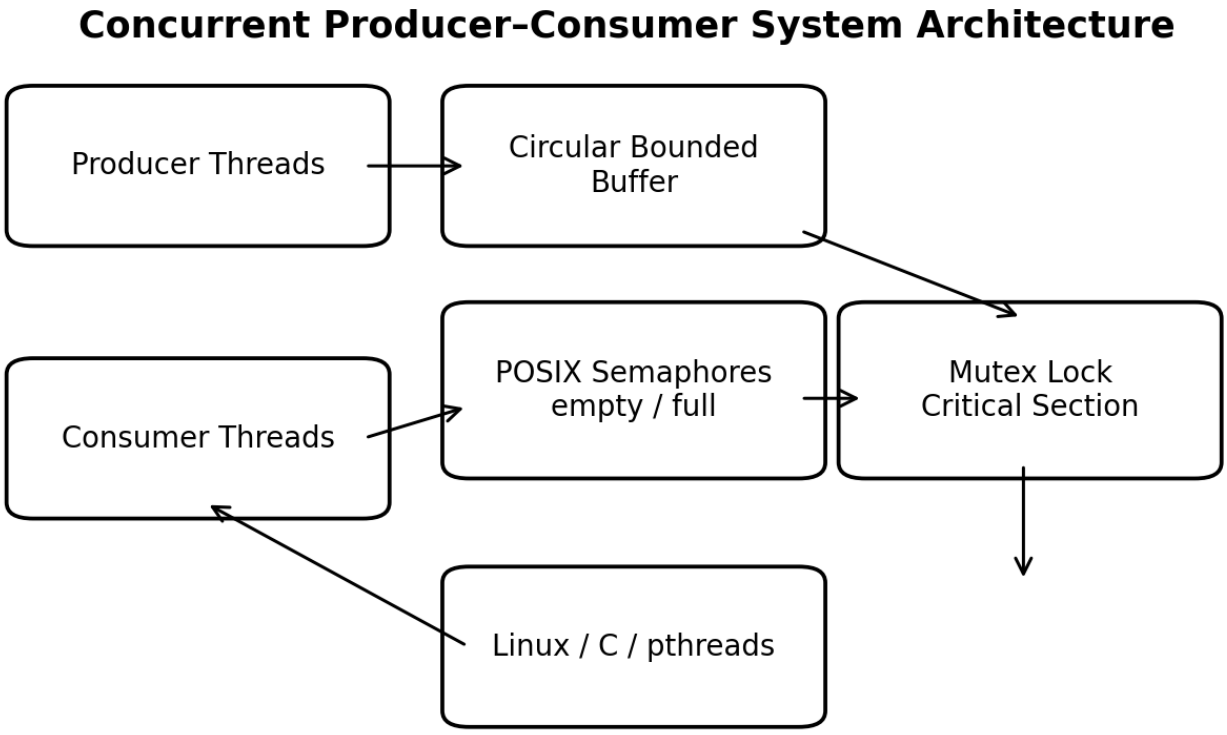
4.6 Linux Commands and Utilities

Command / Utility	Role in the Project
gcc	Compile the C source and link pthread support.
./executable	Run the compiled application.
gdb	Debug thread execution and program behavior.
valgrind --tool=helgrind	Analyze synchronization and potential thread-safety problems.
git / GitHub	Track and share source-code changes.

CHAPTER 5 – SYSTEM ARCHITECTURE AND DESIGN

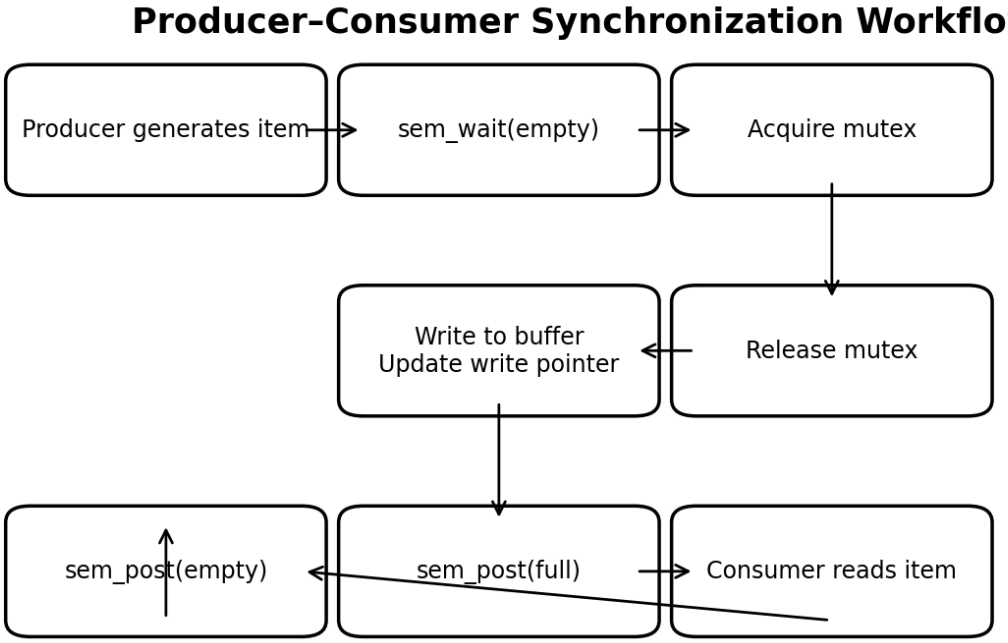
5.1 Overall System Architecture

The system consists of producer threads, consumer threads, a shared circular bounded buffer, counting semaphores, and a mutex. The semaphores coordinate whether a thread is allowed to proceed based on buffer occupancy, while the mutex protects the actual shared-memory update.



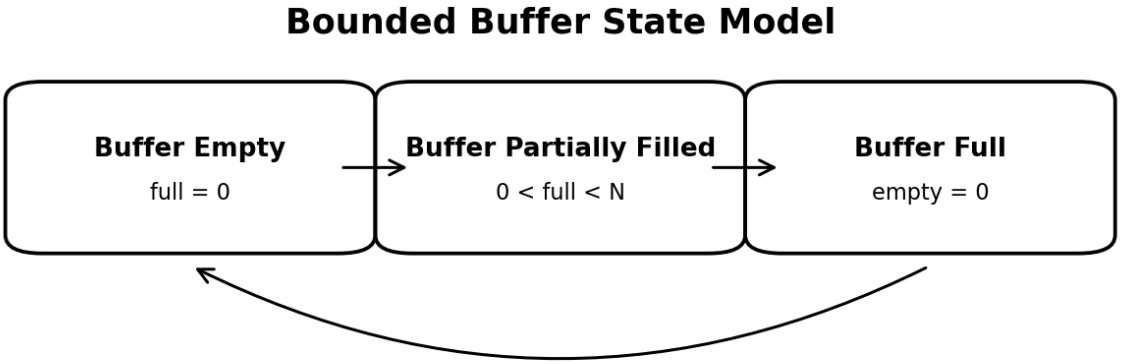
5.2 System Workflow

At initialization, the circular buffer and synchronization objects are created. The empty semaphore is initialized to the buffer capacity N, full is initialized to zero, and the mutex is initialized for exclusive access. Producer and consumer threads are then created.



5.3 Buffer State Model

The buffer can be viewed in three conceptual states: empty, partially filled, and full. The empty and full semaphore counts represent these states and determine which side is permitted to proceed.



5.4 Module Description

Module	Main Responsibility
Buffer Initialization	Allocate the fixed-size circular array and initialize head/tail tracking and synchronization objects.
Producer Workflow	Generate an item, wait for an empty slot, enter the critical section, insert the item, and signal that a slot is full.
Consumer Workflow	Wait for an available item, enter the critical section, remove the item, and signal that an empty slot is available.
Thread Management	Create worker threads, execute concurrent workloads, join completed threads, and perform cleanup.
Synchronization	Coordinate buffer capacity and protect shared-memory operations.
Testing / Validation	Check full and empty states, thread synchronization, throughput, and deadlock avoidance.

5.5 Input and Output Design

The abstract does not define a formal external user-input interface. The primary inputs to the concurrent system are generated work items and the configured producer/consumer workload. The expected outputs include synchronized execution logs, correct buffer state transitions, and evidence of successful concurrent production and consumption.

CHAPTER 6 – IMPLEMENTATION

6.1 Module-wise Implementation

The implementation follows the methodology in the abstract: initialize the bounded buffer and synchronization primitives, create producer and consumer threads, coordinate their access with semaphores and a mutex, join the threads, and release synchronization resources.

6.2 Algorithm / Pseudocode – Initialization

```
BEGIN
  Allocate circular buffer of capacity N
  Set read_index = 0
  Set write_index = 0
  Initialize empty semaphore to N
  Initialize full semaphore to 0
  Initialize mutex
  Create producer and consumer threads
  Wait for all worker threads using pthread_join()
  Destroy/release synchronization resources
END
```

6.3 Algorithm / Pseudocode – Producer

```
PRODUCER()
  repeat for assigned workload
    generate item
    sem_wait(empty)
    lock(mutex)
      buffer[write_index] = item
      update write_index circularly
    unlock(mutex)
    sem_post(full)
  end repeat
```

6.4 Algorithm / Pseudocode – Consumer

```
CONSUMER()
  repeat for assigned workload
    sem_wait(full)
    lock(mutex)
      item = buffer[read_index]
      update read_index circularly
    unlock(mutex)
    sem_post(empty)
    process item
  end repeat
```

6.5 Linux System Calls / APIs Used

API / Concept	Role
pthread_create()	Creates producer and consumer worker threads.
sem_init()	Initializes POSIX semaphore objects.
sem_wait()	Blocks until the semaphore count permits the operation.
sem_post()	Signals that a resource/state change is available.
pthread_mutex_lock()	Enters the protected critical section.
pthread_mutex_unlock()	Leaves the protected critical section.
sleep() / usleep()	Can simulate workload timing and execution latency.
printf()	Formats execution logs and status information.

6.6 Process / Thread Implementation

The project is a multithreaded application rather than a multi-process shared-memory system. Producer and consumer workers are represented as pthreads within the same application address space, allowing them to share the circular buffer and synchronization objects directly.

6.7 Synchronization Implementation

Synchronization has two layers. First, empty/full counting semaphores enforce the buffer-capacity constraint. Second, the mutex ensures that only one thread changes the shared buffer and its associated pointers at a time. The separation of these responsibilities makes the workflow easier to reason about and test.

6.8 Memory Management and Cleanup

The buffer and synchronization objects must remain valid for the lifetime of the worker threads. After pthread_join() confirms that the workers have completed, the application can release the allocated buffer and destroy the semaphore/mutex resources. The abstract explicitly includes cleanup of semaphore and mutex resources.

CHAPTER 7 – EXECUTION AND DEMONSTRATION

7.1 Compilation in Ubuntu / Linux

```
gcc <source-file>.c -o <executable> -lpthread
```

The exact source-file and executable names depend on the team's implementation. The project abstract specifies GCC with pthread linkage, so the final implementation should preserve that compilation requirement.

7.2 Sample Execution Sequence

A typical demonstration should show producer and consumer activity together with the state of the bounded buffer. The exact output text is implementation-dependent and is not supplied in the abstract.

```
Producer: generated item
Producer: waiting for empty slot
Producer: inserted item
Consumer: waiting for full slot
Consumer: removed item
Buffer state: synchronized
...
```

7.3 Test Cases

Test Case	Condition	Expected Observation
TC-01	Buffer initially empty	Consumer blocks on full until a producer inserts an item.
TC-02	Buffer becomes full	Producer blocks on empty until a consumer removes an item.
TC-03	Multiple producers	Mutex prevents simultaneous modification of the shared buffer.
TC-04	Multiple consumers	Mutex prevents simultaneous modification of the shared buffer.
TC-05	High concurrent workload	Application remains synchronized without buffer boundary violations.
TC-06	Thread completion	pthread_join() ensures worker completion before cleanup.

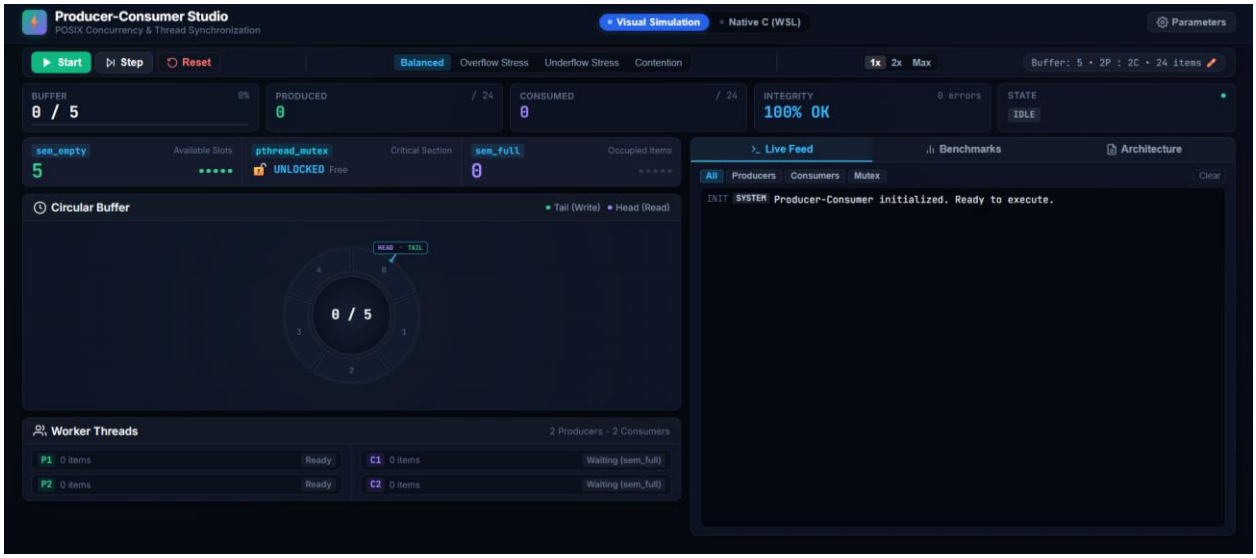
7.4 Demonstration of Operating-System Concepts

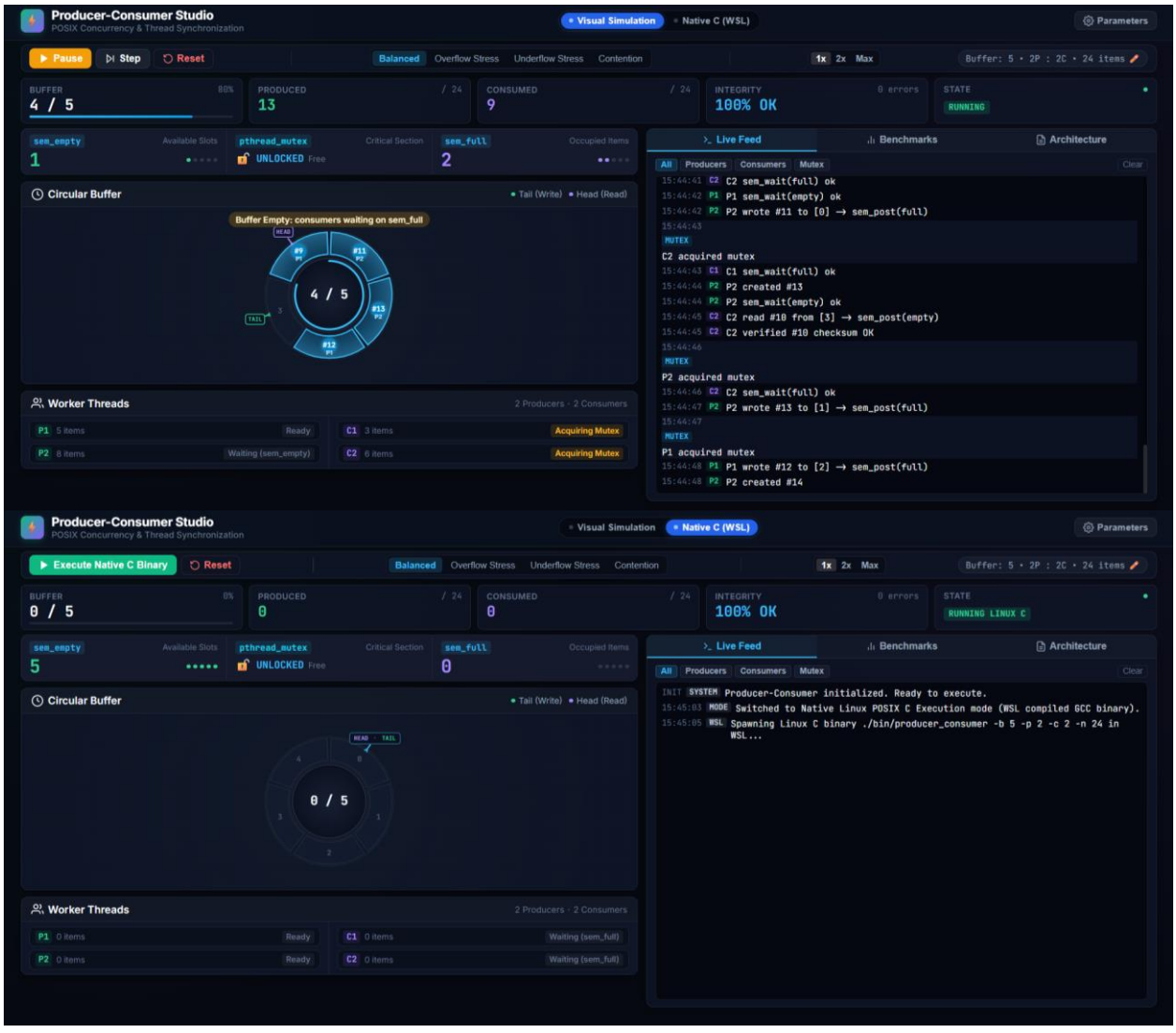
The demonstration should make the relationship between operating-system concepts and observed behavior explicit: sem_wait() demonstrates blocking, sem_post() demonstrates signaling, the mutex demonstrates mutual exclusion, pthread_create() demonstrates concurrency, and the circular buffer demonstrates bounded shared storage.

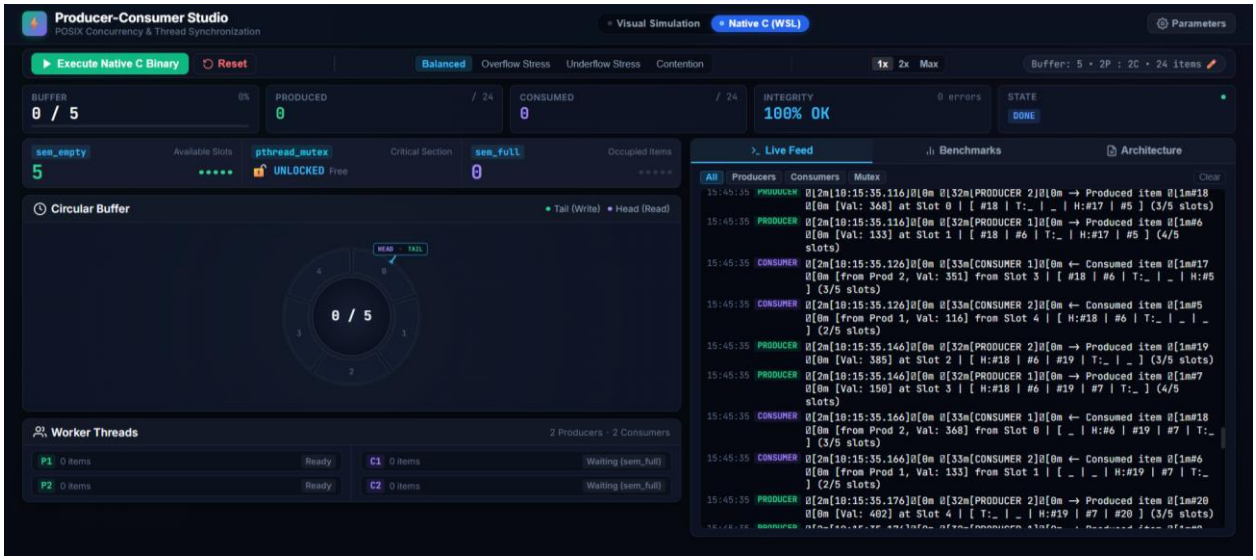
7.5 Validation and Debugging

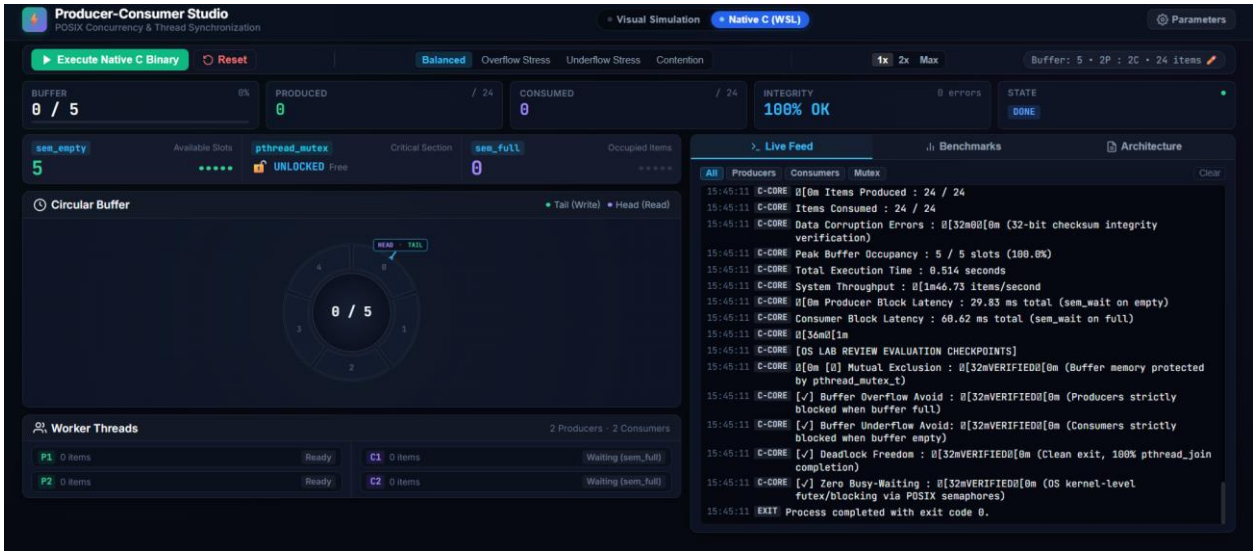
The project material identifies GDB and Valgrind/Helgrind for debugging and synchronization verification. Validation should include checking empty/full boundary conditions, race-condition behavior, thread completion, and consistency of produced/consumed data.

7.6 Screenshots

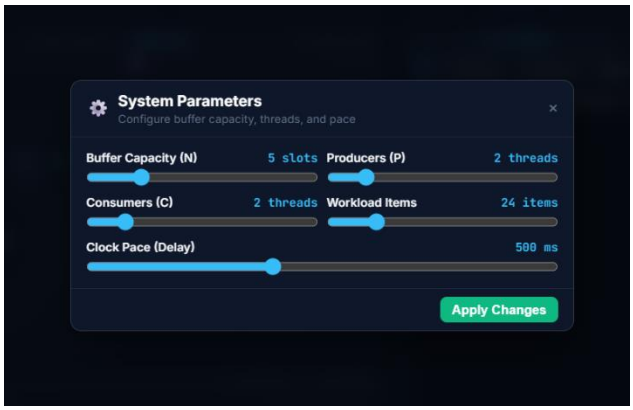





Producer-Consumer Studio
POSIX Concurrency & Thread Synchronization
Visual Simulation
Native C (WSL)
Parameters
Execute Native C Binary
Reset
Balanced
Overflow Stress
Underflow Stress
Contention
1x
2x
Max
Buffer: 5 • 2P : 2C • 24 items
BUFFER
0 / 5
PRODUCED
0
/ 24
CONSUMED
0
/ 24
INTEGRITY
100% OK
0 errors
STATE
DONE
sem_empty
Available Slots
5
pthread_mutex
Critical Section
UNLOCKED Free
sem_full
Occupied Items
0
Live Feed
Benchmarks
Architecture
All
Producers
Consumers
Mutex
Clear
Circular Buffer
Tail (Write)
Head (Read)
HEAD · TAIL
0 / 5
Worker Threads
2 Producers · 2 Consumers
P1 0 items
Ready
P2 0 items
Ready
C1 0 items
Waiting (sem_full)
C2 0 items
Waiting (sem_full)


Producer-Consumer Studio
POSIX Concurrency & Thread Synchronization
Visual Simulation
Native C (WSL)
Parameters
Execute Native C Binary
Reset
Balanced
Overflow Stress
Underflow Stress
Contention
1x
2x
Max
Buffer: 5 • 2P : 2C • 24 items
BUFFER
0 / 5
PRODUCED
0
/ 24
CONSUMED
0
/ 24
INTEGRITY
100% OK
0 errors
STATE
DONE
sem_empty
Available Slots
5
pthread_mutex
Critical Section
UNLOCKED Free
sem_full
Occupied Items
0
Live Feed
Benchmarks
Architecture
All
Producers
Consumers
Mutex
Clear
15:45:11 C-CORE Items Produced : 24 / 24
15:45:11 C-CORE Items Consumed : 24 / 24
15:45:11 C-CORE Data Corruption Errors : 0 (32-bit checksum integrity verification)
15:45:11 C-CORE Peak Buffer Occupancy : 5 / 5 slots (100.0%)
15:45:11 C-CORE Total Execution Time : 0.514 seconds
15:45:11 C-CORE System Throughput : 1m46.73 items/second
15:45:11 C-CORE 0m Producer Block Latency : 29.83 ms total (sem_wait on empty)
15:45:11 C-CORE Consumer Block Latency : 60.62 ms total (sem_wait on full)
15:45:11 C-CORE 36m1m
15:45:11 C-CORE [OS LAB REVIEW EVALUATION CHECKPOINTS]
15:45:11 C-CORE 0m [0] Mutual Exclusion : 32mVERIFIED0m (Buffer memory protected by pthread_mutex_t)
15:45:11 C-CORE [✓] Buffer Overflow Avoid : 32mVERIFIED0m (Producers strictly blocked when buffer full)
15:45:11 C-CORE [✓] Buffer Underflow Avoid: 32mVERIFIED0m (Consumers strictly blocked when buffer empty)
15:45:11 C-CORE [✓] Deadlock Freedom : 32mVERIFIED0m (Clean exit, 100% pthread_join completion)
15:45:11 C-CORE [✓] Zero Busy-Waiting : 32mVERIFIED0m (OS kernel-level futex/blocking via POSIX semaphores)
15:45:11 EXIT Process completed with exit code 0.
Circular Buffer
Tail (Write)
Head (Read)
HEAD · TAIL
0 / 5
Worker Threads
2 Producers · 2 Consumers
P1 0 items
Ready
P2 0 items
Ready
C1 0 items
Waiting (sem_full)
C2 0 items
Waiting (sem_full)


/ 24
INTEGRITY
100% OK
0 errors
STATE
IDLE
Occupied Items
(Write) • Head (Read)
Live Feed
Benchmarks
Architecture
Throughput Benchmark
Operations per second across thread contention ratios (Helgrind verified 0 race conditions).
Run C Benchmark
1P : 1C (Baseline Serial) 342,912 items/sec
1P : 4C (Consumer Heavy / Underflow) 119,021 items/sec
4P : 1C (Producer Heavy / Overflow) 121,517 items/sec
4P : 4C (Balanced Concurrency) 519,697 items/sec
8P : 8C (High Concurrency) 566,476 items/sec
16P : 16C (Stress / Peak Contention) 714,898 items/sec
SCENARIO
BUFFER
TIME
THROUGHPUT
INTEGRITY
1P : 1C (Baseline Serial) 10 0.0292 s 342,912 /s 0 Errors (Verified)
1P : 4C (Consumer Heavy / Underflow) 10 0.0840 s 119,021 /s 0 Errors (Verified)
4P : 1C (Producer Heavy / Overflow) 10 0.0823 s 121,517 /s 0 Errors (Verified)
4P : 4C (Balanced Concurrency) 10 0.0192 s 519,697 /s 0 Errors (Verified)
8P : 8C (High Concurrency) 20 0.0177 s 566,476 /s 0 Errors (Verified)
16P : 16C (Stress / Peak Contention) 32 0.0140 s 714,898 /s 0 Errors (Verified)
Waiting (sem_full)
Waiting (sem_full)


System Parameters
Configure buffer capacity, threads, and pace
Buffer Capacity (N) 5 slots
Producers (P) 2 threads
Consumers (C) 2 threads
Workload Items 24 items
Clock Pace (Delay) 500 ms
Apply Changes

CHAPTER 8 – CONCLUSION

8.1 Summary of the Project

The project presents a concurrent Producer-Consumer system implemented in C for Linux using POSIX threads, POSIX semaphores, a mutex, and a circular bounded buffer. The design addresses the core synchronization problems associated with concurrent access to shared storage.

8.2 Objectives Achieved

- A thread-safe circular bounded-buffer design is specified.
- Counting semaphores coordinate empty and full buffer capacity.
- Mutex-based mutual exclusion protects the critical section.
- The design uses blocking synchronization rather than CPU-intensive busy-waiting.
- The methodology includes testing for synchronization, deadlock avoidance, and throughput.

8.3 Expected Results

The expected outcome stated in the project material is a functional, compiled Linux command-line application that demonstrates concurrent data production and consumption, prevents race conditions and buffer-boundary violations, avoids busy-waiting through blocking synchronization, and produces synchronized execution logs for empty/full buffer conditions.

8.4 Skills and Learning Outcomes

- Understanding of concurrency and thread lifecycle management.
- Practical use of POSIX semaphores and mutex locks.
- Design of a bounded circular buffer for shared data.
- Recognition and prevention of race conditions and deadlock-related problems.
- Linux compilation, execution, debugging, and synchronization testing.
- Use of version control and collaborative source-code management.

8.5 Limitations and Future Work

The supplied abstract does not define additional application features beyond the core Producer-Consumer synchronization system. Future iterations can therefore focus on richer benchmarking, configurable thread counts and buffer sizes, detailed performance visualization, and extended stress testing, provided these are added to the actual implementation and documented with measured results.

CHAPTER 9 – REFERENCES

9.1 References

1. The Open Group. POSIX.1 — Base Specifications: Threads.
https://pubs.opengroup.org/onlinepubs/9799919799/basedefs/V1_chap02.html
2. Linux man-pages Project. pthreads(7) — POSIX Threads.
https://man7.org/linux/man-pages/man7/pthreads.7.html
3. Linux man-pages Project. sem_overview(7) — Overview of POSIX Semaphores.
https://man7.org/linux/man-pages/man7/sem_overview.7.html
4. Linux man-pages Project. sem_wait(3) — Lock a Semaphore.
https://man7.org/linux/man-pages/man3/sem_wait.3.html
5. Linux man-pages Project. sem_post(3) — Unlock a Semaphore.
https://man7.org/linux/man-pages/man3/sem_post.3.html
6. Linux man-pages Project. pthread_create(3) — Create a New Thread
https://man7.org/linux/man-pages/man3/pthread_create.3.html
7. Linux man-pages Project. pthread_mutex_lock(3) — Lock and Unlock a Mutex.
https://man7.org/linux/man-pages/man3/pthread_mutex_lock.3p.html
8. Silberschatz, Galvin, P. B., and Gagne, G. Operating System Concepts. 10th ed., Wiley, 2018.
9. Tanenbaum, A. S., and Bos, H. Modern Operating Systems. 4th ed., Pearson, 2014.
10. Valgrind Developers. Valgrind User Manual.
https://valgrind.org/docs/manual/manual.html